Study Guide

Read this 3 times before you walk in

Mohammad Kamrul Hasan · AI-Augmented QA Architect · April 2026

YOUR AUDIENCE

Your team and your boss have **heard** about AI agents. They have not **seen** one work in practice on real company data. That gap is your opening. They walk in skeptical or curious. You need them to walk out thinking *we have one of these now, and Mohammad built it.*

Lead with working software, not theory. First thing they should see is the agent doing something visible whose result is verifiable in our actual systems (GitLab, email inbox, memory DB). Theory comes after.

YOUR OPENING LINE

"You have all heard about AI agents this past year. The Anthropic and OpenAI announcements, the demos, the LinkedIn posts. Tonight you are going to see one running on our infrastructure, on our data, doing real work for our team. Not a slide deck about what AI might do. A working agent that filed thirteen real client requirements into our GitLab backlog this morning, from emails and meeting recordings, while we slept. I am Mohammad. I am a QA architect by trade. I designed and shipped this end-to-end by directing AI through Claude Code. Let me show you what that looks like."

THE THESIS YOU ARE PROVING

"A QA architect who knows how to direct AI now ships production infrastructure that previously required a senior developer team. This is not a marginal productivity gain. It is a different kind of professional. NAPCO Nucleus is the proof."

Stay on this thesis the entire talk. Every demo step, every architecture choice, every hard answer anchors back to it. **QA architect plus AI equals senior dev team output.**

NUMBERS TO KNOW COLD

If asked any of these, answer instantly without looking down.

9	31	4	2	$200
WORKFLOWS	MCP TOOLS	TESTS AT 2AM	DAILY EMAILS	PER MONTH

ARCHITECTURE IN ONE PARAGRAPH (memorize this)

NAPCO Nucleus is an AI agent built on the Claude Agent SDK that runs as nine scheduled GitHub Actions workflows on a self-hosted Windows runner. It serves two operational dimensions for the MVP Access engineering org: **Project Management** (ingest client requirements from email and Drive recordings, split into 3-hour tasks, publish to GitLab with two-layer dedup) and **Test Automation** (orchestrate the 4 test suites at 02:00 BDT every night, then ship two consolidated emails the next morning at 09:00 and 09:30). Reasoning runs on the local Claude Code CLI under a Claude Max subscription. State persists in a committed SQLite database with FTS5. 31 MCP tools are exposed. Real production deployment, not a prototype.

THE 6 ENGINEERING DECISIONS (each with a why)

- **Claude Max via local CLI, not API key.** Reasoning is essentially free at the margin. Eight scheduled workflows fire 30 times a day. On the API that would be real money. Fixed monthly cost wins clearly at our scale.
- **Algorithms in prompts, not Python.** Tools wrap I/O only. Classification, summarization, regression analysis happen in markdown that Claude executes. Behavior changes by editing markdown, not Python.
- **Memory committed to git.** SQLite + FTS5 in the repo. Clone the project, you have the agent's full history. Disaster recovery is git clone. Zero infrastructure to provision.
- **Self-hosted runner.** Direct access to staging behind VPN, direct access to TFS, direct access to IIS UNC paths. None reachable from a hosted runner without complex tunneling. Secrets stay on our infra.
- **One Claude turn per process.** agent.py loads, runs ONE turn, exits. Clean process isolation. Memory is the explicit connection between turns.
- **Two consolidated emails per day, not one per workflow.** 09:00 BDT detailed test report to the team. 09:30 BDT executive summary to leadership. Higher signal-to-noise than six fragments scattered across the night.

THE 9 WORKFLOWS (know each one)

All 4 tests fire at 02:00 BDT. Reports go out at 09:00 and 09:30 BDT.

#	Workflow	Schedule	What it does
1	API Functional Test	02:00 BDT	Newman / Postman across the API surface.
2	API Integration Test	02:00 BDT	pytest with regression diff.
3	API Load Test	02:00 BDT	Locust 5 tiers, 10 to 10K users with cooldowns.
4	MVP Access E2E Test	02:00 BDT	Playwright full suite. Screenshots on failure.
5	Daily Report Detailed	09:00 BDT	4-test detailed PDF to the FULL TEAM.
6	Daily Report Summary	09:30 BDT	6-block dashboard to LEADERSHIP.
7	Requirement Management	Every 2h biz hrs	IMAP + Drive ingest. Files in GitLab.
8	MVPAccess CICD	22:00 BDT	TFS pull, MSBuild, IIS deploy via UNC.
9	Probe Runner Filesystem	Manual	Diagnostic for runner triage.

VOCABULARY (use these words confidently)

- **MCP**: Model Context Protocol. Anthropic's spec for tool integration. Each NN tool is wrapped with @tool() and registered with the MCP server. The Claude CLI discovers and calls them.
- **Claude Agent SDK**: Anthropic's Python library for building agents. ClaudeAgentOptions, ClaudeSDKClient, create_sdk_mcp_server.
- **FTS5**: SQLite full-text search. Enables fuzzy matching so 'add SSO to staging' is caught as a duplicate of 'staging Azure SSO integration'.
- **Self-hosted runner**: A GitHub Actions runner installed on your own machine. Required for access to private resources (staging, TFS, IIS) that hosted runners cannot reach.
- **Idempotency / dedup**: Running the same operation twice has the same effect as once. NN's requirement-management is idempotent via two-layer check: title-match against open issues plus FTS5 fuzzy-match against requirements_seen.
- **Whisper**: Speech-to-text. NN uses Groq's Whisper API to transcribe meeting recordings dropped in the Drive folder.
- **Override semantics**: load_dotenv(override=True) means values in .env take precedence over inherited shell env. Local .env beats stale GHA secret values.

HARD QUESTIONS - PART 1 (rehearse out loud)

Q. Did you write all this Python yourself? Are you really a developer?

I am a QA architect, not a developer by trade. That is the whole point of NAPCO Nucleus. I designed the system: every workflow, every prompt, every architectural decision, every trade-off. The Python implementation was AI-assisted. I directed Claude through Claude Code CLI to write the code under my specs. That is the new model. A QA architect who directs AI ships production infrastructure that previously required a senior dev team. The GitLab issues are real. The reports are real. The deploy pipeline is real. They exist because I built the system. The language they are written in happens to be Python via AI partnership.

Q. Why not just use GitHub Copilot, Cursor, or ChatGPT?

Those are interactive coding assistants. A human types, the AI suggests. NN is the opposite. It runs unattended on a schedule, reads its own state from memory, takes actions, ships artifacts. No human in the loop per run. Different tool category.

Q. What if the AI makes a mistake? Wrong classification, wrong issue title?

Two layers protect us. First, every action is auditable: activity_logs in memory, GHA run logs, git commits, GitLab issue history. We can trace what the agent did and why. Second, mutating tools (publish_tasks_to_gitlab, send_email_report) all support a dry_run mode. We use dry-run extensively in development. Mistakes happen but they are visible and reversible.

Q. How much does this cost?

About $200 per month for the Claude Max subscription. Fixed regardless of how many times the agent runs. Self-hosted runner is on existing hardware, zero added cost. Groq Whisper is metered but cheap. GitLab and Google use existing seats. Compared to API-key billing where 30 daily runs would cost meaningfully more, the Claude Max model wins clearly at our scale.

HARD QUESTIONS - PART 2

Q. Is our data safe? Where does it go?

Emails read from our Gmail via IMAP (app password we control). Drive recordings transcribed via Groq (audio sent, transcript returned, audio not retained per their policy). Text reasoning sent to Anthropic via the Claude CLI under our Max subscription terms. Memory database stays on our self-hosted runner and in our git repo. Nothing leaves our control except for the LLM reasoning calls and audio transcription. Both are commercial subscriptions with explicit data-handling terms.

Q. What happens if Claude is down?

The workflow fails cleanly with the API error message logged to GHA. The next scheduled run picks up where the failed one left off because memory is checkpoint-based. Daily Report degrades gracefully: if it cannot compose a fresh executive summary, it sends a shorter status email noting the outage.

Q. Why not a deterministic Python script instead of an AI?

We had one. 1,961 lines of regex heuristics and hardcoded if-else trees in tools_legacy.py. Maintaining it meant editing Python every time a new failure pattern appeared. The new design moves classification into prompts. New failure category? Markdown edit, no code review. The agent also does things deterministic code cannot: reading a 30-page meeting transcript and extracting actionable requirements.

Q. How do you debug it when something goes wrong?

Three levels. (1) GHA UI shows every workflow run with full stdout and stderr. (2) memory.recall_activity() lets me query what the agent did in the last X hours for task Y. (3) prompts/ are version-controlled markdown. I can read exactly what Claude was instructed to do at any point in history.

HARD QUESTIONS - PART 3

Q. What if the agent runs in a loop or runs up costs?

Three guardrails. (1) GHA workflow timeouts (15-60 min per workflow). (2) Tool descriptions explicitly say AT MOST ONCE per run for expensive operations. (3) Claude Max is fixed monthly cost, so a runaway loop cannot surprise us with a bill.

Q. Can you explain why the agent made a particular decision?

Yes. Every Claude turn is logged by GHA (full reasoning trace plus tool call sequence). Every tool call's args and result are in activity_logs. For any GitLab issue NN created, I can show you the original email or transcript text, the prompt that asked Claude to split it, and the JSON of tasks Claude proposed.

Q. How do you know it is doing useful work and not theater?

Concrete proof: 13 GitLab issues created today from email plus meeting recordings, each linked back to source. 27 backend test failures classified into known-bug, regression, flaky, data-issue buckets. The load test surfaced staging strain at 100 users, a real performance signal we acted on. None of these are demoware. They are in our actual GitLab and our actual reports.

Q. What is still rough?

Three things. (1) The GHA self-hosted runner currently runs as a Windows user that cannot see the sibling test project trees. A permission fix is needed. (2) The CICD workflow needs IT to add 6 secrets (TFS plus IIS) before it can actually deploy. The YAML is production-ready, the credentials are not there. (3) E2E test stability. Playwright is sensitive to timing on staging. We have flaky tests we are working through. Roadmap addresses all three.

HOW TO LOOK CREDIBLE - PRESENTATION TACTICS

- **Pre-flight 30 minutes before.** Open the terminal. Have nucleus_memory.db open in DB Browser. Have GitHub repo, GitLab project, Drive folder in browser tabs.
- **Narrate while live tools run.** 'It is polling IMAP first... now downloading from Drive... now Claude is splitting this email into three tasks...' Silent waiting feels broken; narrated waiting feels like a tour.
- **If something breaks, name it immediately.** 'OK this just failed because X. Watch what the agent does. See how it logs the error to memory and exits cleanly?' Failure handled openly is MORE credible than a perfect demo.
- **Pause 2 seconds before answering hard questions.** Looks thoughtful, not slow. Then name the trade-off: 'Good question. We considered Y and chose X because of Z.'
- **Refer to files by name.** 'That is in tools/_shared.py at line 50.' Not 'somewhere in the codebase'. Specificity equals mastery.
- **If you do not know, say 'let me check'.** Open the file, look it up. Far better than guessing.
- **Concrete before abstract.** 'Here are 13 GitLab issues NN created today.' Then: 'Now let me show you HOW.'
- **End with the ask.** If you want IT to grant runner permissions, DevOps to add CICD secrets, or your boss to approve Claude Max as a line item, say so explicitly in the closing 30 seconds.

THE ONE LINE FOR THE BOSS

"NAPCO Nucleus is an AI agent I architected for the MVP Access team. It reads our client emails and meeting recordings every two hours, files them as tasks in GitLab, runs our test suites every night at 02:00, classifies the failures, and emails two consolidated reports each morning. Built on Claude. Runs on our own hardware. Costs about two hundred dollars a month flat. Created thirteen real tasks from this morning's inbox while we slept. I designed it. AI implemented it under my direction. This is what a QA architect can ship now."